

Discover Anything



Start Writing

Log in



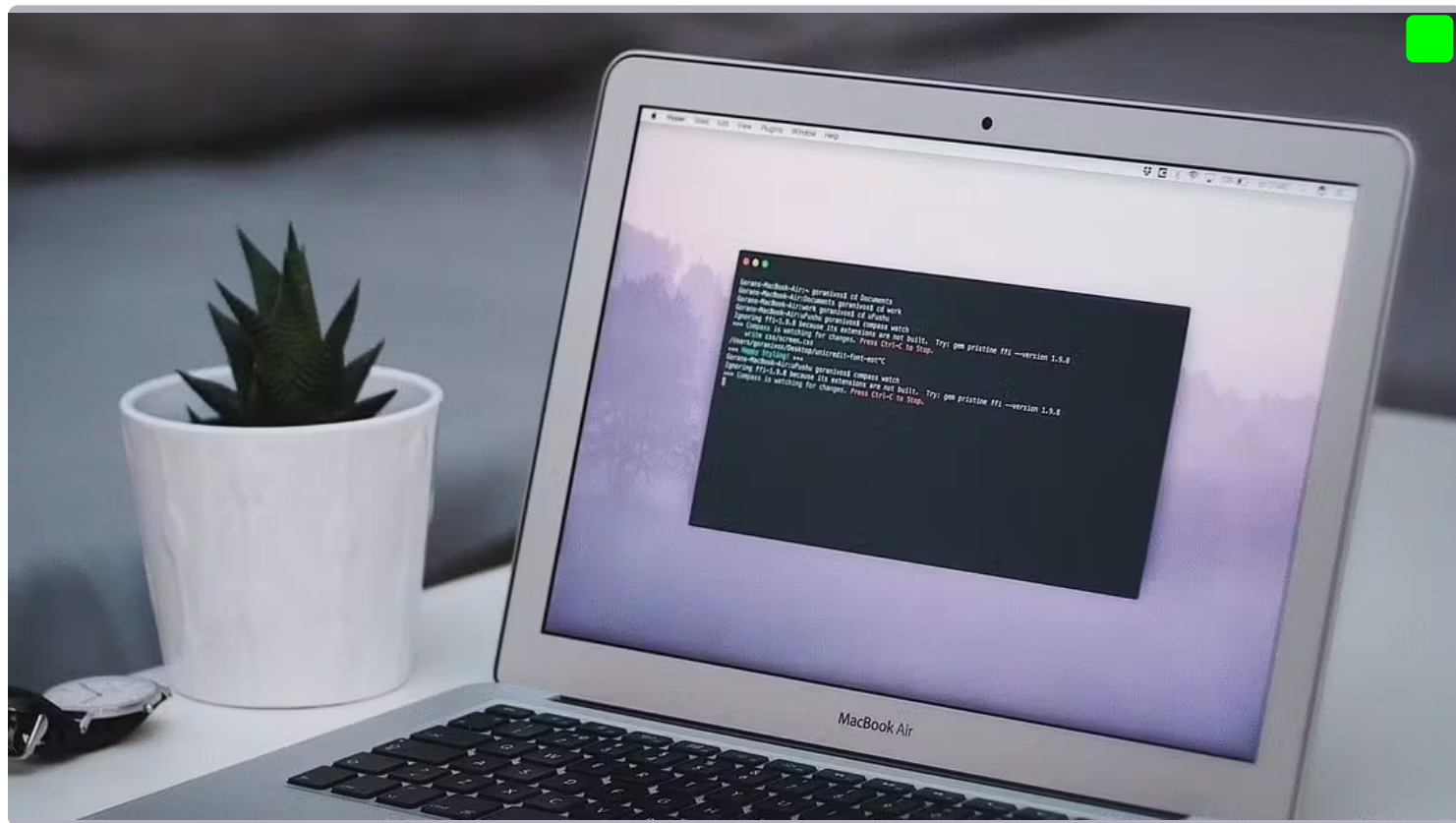
Hands-On Learning From ML Experts



4,411 reads

Creating a Terminal Emulator in React

May 23rd 2022 | by @alexandrutasica 4,411 reads



PROGRAMMING

#react

#reactjs

#terminal

#cli

#react-hooks

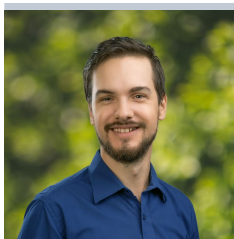
#javascript

#software-development

#software-engineering



1x

Read by **Dr. One**
(en-US)Audio Presented by **@alexandrutasica****Alexandru Tasica***Excited to empower and help clients worldwide, bringing t...**Receive Stories from @alexandrutasica*

name@company.com

SUBSCRIBE

The Terminal window, from back in the days when the computers were operated only by the command line or in the present when developing even the smallest project, we all use it. If this does not ring a bell yet, here is a [definition from Wikipedia](#) about terminals aka. command-line interfaces:

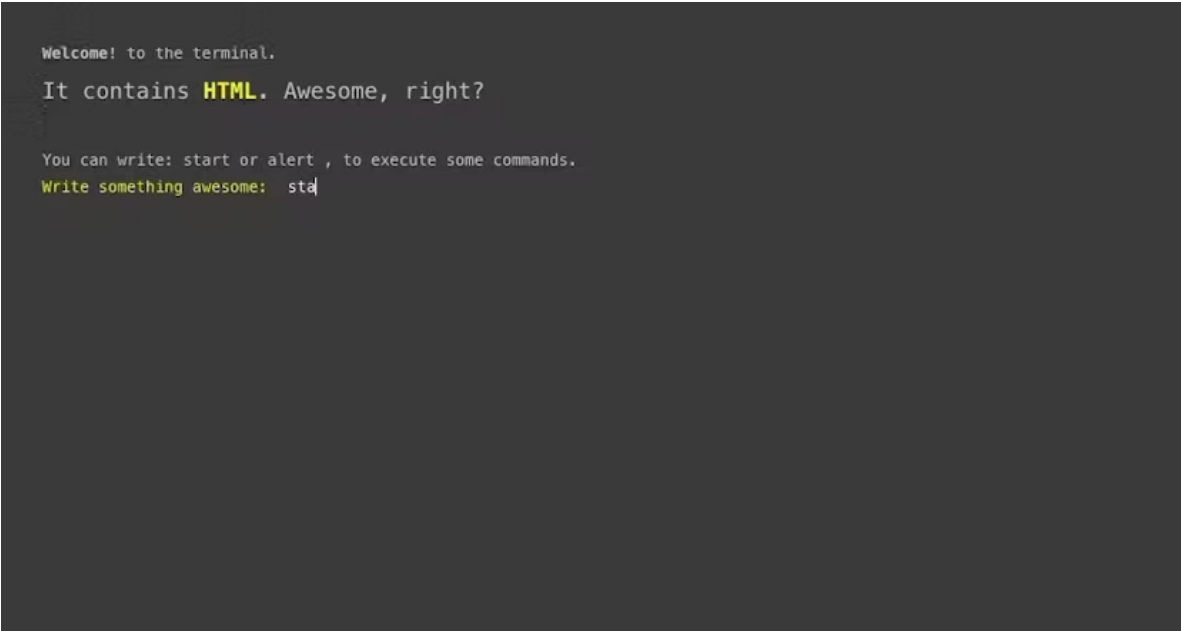
A **command-line interface (CLI)** processes commands to a computer program in the form of lines of text. The program which handles the interface is called a **command-line interpreter** or **command-line processor**.

These terminal interfaces are used by a lot of websites like Haskell, Hyper, even Codesandbox has a terminal (console). Let's make one with [ReactJs](#) for our website.

What we will build

We will build a terminal emulator that accepts a set of predefined commands that we give it. You can [view the code on GitHub](#) or





```
Welcome! to the terminal.  
It contains HTML. Awesome, right?  
You can write: start or alert , to execute some commands.  
Write something awesome: sta
```

To build this terminal, we are using `create-react-app` (CRA) to generate a new React application through our computer terminal. You can even see a terminal window animation on the CRA website in the section “Get started in seconds”.

First things first

First things first, let’s create our React project using the `create-react-app` (CRA) CLI. We will create an application with the `Typescript` template, to make our code better and avoid possible mistakes like adding two strings when we don’t want to.

I will name the application `terminal` so, on my computer console, I will run the following code:

```
npx create-react-app terminal --template typescript
```

After that, I will open the newly created `terminal` folder in my code editor of choice, `WebStorm`.

Let’s clean up the files a little bit. Let’s delete the files `logo.svg`, `App.css` and `App.test.tsx` from the `src/` folder. From the file `App.tsx`, let’s remove everything that is within the `div` with the className `App` and also delete line number 2 and 3, with the logo import. Like so:

```
import React from 'react';
```

```
function App() {  
  return (  
    <div className="App">  
      We will fill this section later  
    </div>  
  );  
}  
  
export default App;
```

Creating the style and basic structure

In the `src/` folder, let's create another folder with the name `Terminal/`, and inside that, create a file called `index.tsx` one called `terminal.css`. This will be our basic structure.

The terminal window is composed of two elements, the output, and the input. The output is where we will write the response, and the input is where we will write our commands. So, based on that, let's create our terminal and add some style to it.

In the `index.tsx`, we will have the following code:

```
import './terminal.css';  
  
export const Terminal = () => {  
  return (  
    <div className="terminal">  
      <div className="terminal__line">A terminal line</div>  
      <div className="terminal__prompt">  
        <div className="terminal__prompt__label">alexandru.tasica:</div>  
        <div className="terminal__prompt__input">  
          <input type="text" />  
        </div>  
      </div>  
    </div>  
  );  
};
```

Our `terminal.css` style will be the following:

```
.terminal {
  height: 500px;
  overflow-y: auto;
  background-color: #3C3C3C;
  color: #C4C4C4;
  padding: 35px 45px;
  font-size: 14px;
  line-height: 1.42;
  font-family: 'IBM Plex Mono', Consolas, Menlo, Monaco, 'Courier New', Courier,
  monospace;
  text-shadow: 0 4px 4px rgba(0, 0, 0, 0.25);
}

.terminal__line {
  line-height: 2;
  white-space: pre-wrap;
}

.terminal__prompt {
  display: flex;
  align-items: center;
}

.terminal__prompt_label {
  flex: 0 0 auto;
  color: #F9EF00;
}

.terminal__prompt_input {
  flex: 1;
  margin-left: 1rem;
  display: flex;
  align-items: center;
  color: white;
}

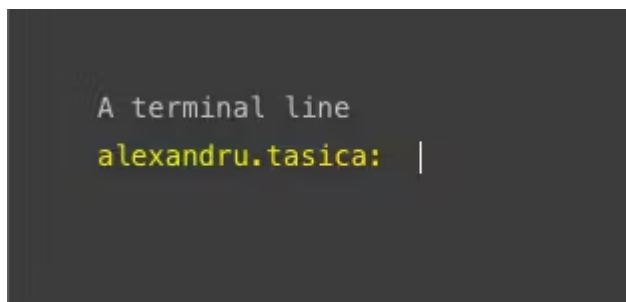
.terminal__prompt_input input {
  flex: 1;
  width: 100%;
  background-color: transparent;
  color: white;
  border: 0;
  outline: none;
  font-size: 14px;
  line-height: 1.42;
  font-family: 'IBM Plex Mono', Consolas, Menlo, Monaco, 'Courier New', Courier,
```

```
monospace;  
}
```

And also, let's import that in our `App.tsx` to view it in our browser:

```
import React from 'react';  
import {Terminal} from './Terminal';  
  
function App() {  
  return (  
    <div className="App">  
      <Terminal />  
    </div>  
  );  
}  
  
export default App;
```

And the result is, 🥁 drums please:



Good, so you've got where we are heading to.

Basic state management

Awesome! So we have the basic structure. Now we need to make users interact with it, keep our messages history and add new messages to the terminal.

For that, we will write a custom hook called `useTerminal` that will go hand-in-hand with our terminal. Let's see the basic hook structure that we are going for:

```
import {useCallback, useEffect, useState} from 'react';  
import {TerminalHistory, TerminalHistoryItem, TerminalPushToHistoryWithDelayProps} from  
"./types";
```

```

export const useTerminal = () => {
  const [terminalRef, setDomNode] = useState<HTMLDivElement>();
  const setTerminalRef = useCallback((node: HTMLDivElement) => setDomNode(node), []);

  const [history, setHistory] = useState<TerminalHistory>([]);

  /**
   * Scroll to the bottom of the terminal when window is resized
   */
  useEffect(() => {
    const windowResizeEvent = () => {
      terminalRef?.scrollTo({
        top: terminalRef?.scrollHeight ?? 99999,
        behavior: 'smooth',
      });
    };
    window.addEventListener('resize', windowResizeEvent);

    return () => {
      window.removeEventListener('resize', windowResizeEvent);
    };
  }, [terminalRef]);

  /**
   * Scroll to the bottom of the terminal on every new history item
   */
  useEffect(() => {
    terminalRef?.scrollTo({
      top: terminalRef?.scrollHeight ?? 99999,
      behavior: 'smooth',
    });
  }, [history, terminalRef]);

  const pushToHistory = useCallback((item: TerminalHistoryItem) => {
    setHistory((old) => [...old, item]);
  }, []);

  /**
   * Write text to terminal
   * @param content The text to be printed in the terminal
   * @param delay The delay in ms before the text is printed
   * @param executeBefore The function to be executed before the text is printed
   * @param executeAfter The function to be executed after the text is printed
   */
  const pushToHistoryWithDelay = useCallback(

```



```

    ({
      delay = 0,
      content,
    }: TerminalPushToHistoryWithDelayProps) =>
      new Promise((resolve) => {
        setTimeout(() => {
          pushToHistory(content);
          return resolve(content);
        }, delay);
      }),
    [pushToHistory]
  );

  /**
   * Reset the terminal window
   */
  const resetTerminal = useCallback(() => {
    setHistory([]);
  }, []);

  return {
    history,
    pushToHistory,
    pushToHistoryWithDelay,

    terminalRef,
    setTerminalRef,

    resetTerminal,
  };
};

```

And let's also create a separate file that contains the type of the props and all the things we want to strongly type, like `Terminal History`. Let's create a file called `types.ts` with the definitions we have right now.

```

import { ReactNode } from "react";

export type TerminalHistoryItem = ReactNode | string;
export type TerminalHistory = TerminalHistoryItem[];
export type TerminalPushToHistoryWithDelayProps = {
  content: TerminalHistoryItem;
  delay?: number;
};

```

Now, after we have the types defined, let's see a quick description of what does every function in our hook:

- The first state, `terminalRef` will keep our reference to the terminal container. We will reference it later in our `terminal/index.tsx` file. The first function is a helper function that sets the reference for that `div`.
- The first two `useEffects` will scroll down the terminal every time there is a new entry in our terminal history or every time the window is resized
- `pushToHistory` will push a new message to our terminal history
- `pushToHistoryWithDelay` will push the new message to our terminal history with a specific delay. We return a promise instead of resolving it as a function, so we can chain multiple pushes as an animation.
- Last, we have the `resetTerminal` which resets the terminal history. Works more like a `clear` function from the classic terminal.

Awesome! Now that we have a way to store and push the messages, let's go integrate this hook with structure. In the `Terminal/index.tsx` In the file, we will have some props that will be connected with the terminal history and also will handle the focus on our terminal input prompt.

```
import './terminal.css';
import {ForwardedRef, forwardRef, useCallback, useEffect, useRef} from "react";
import {TerminalHistory, TerminalHistoryItem} from "./types";

export type TerminalProps = {
  history: TerminalHistory;
  promptLabel?: TerminalHistoryItem;
};

export const Terminal = forwardRef(
  (props: TerminalProps, ref: ForwardedRef<HTMLDivElement>) => {
    const {
      history = [],
      promptLabel = '>',
    } = props;

    /**
     * Focus on the input whenever we render the terminal or click in the terminal
     */
  })
```

```

    */
    const inputRef = useRef<HTMLInputElement>();
    useEffect(() => {
      inputRef.current?.focus();
    });

    const focusInput = useCallback(() => {
      inputRef.current?.focus();
    }, []);

    return (
      <div className="terminal" ref={ref} onClick={focusInput}>
        {history.map((line, index) => (
          <div className="terminal__line" key={`terminal-line-${index}-${line}`}>
            {line}
          </div>
        ))}
        <div className="terminal__prompt">
          <div className="terminal__prompt_label">{promptLabel}</div>
          <div className="terminal__prompt_input">
            <input
              type="text"
              // @ts-ignore
              ref={inputRef}
            />
          </div>
        </div>
      </div>
    );
  });

```

And not, let's connect the dots and push something to the terminal. Because we accept any `ReactNode` as a line, we can push any HTML we want.

Our `App.tsx` will be the wrapper, this is where all the action will happen. Here we will set the messages we want to show. Here is a working example:

```

import React, {useEffect} from 'react';
import {Terminal} from "../Terminal";
import {useTerminal} from "../Terminal/hooks";

function App() {
  const {
    history,
    pushToHistory,
    setTerminalRef,

```

```

    resetTerminal,
  } = useTerminal();

  useEffect(() => {
    resetTerminal();

    pushToHistory(<>
      <div><strong>Welcome!</strong> to the terminal.</div>
      <div style={{fontSize: 20}}>It contains <span style={{color: 'yellow'}}>
<strong>HTML</strong></span>. Awesome, right?</div>
    </>
  );
}, []);

return (
  <div className="App">
    <Terminal
      history={history}
      ref={setTerminalRef}
      promptLabel={<>Write something awesome:</>}
    />
  </div>
);
}

export default App;

```

And, the result so far is:



On mount, we just push whatever we want, like that cool big yellow HTML text.

Implementing commands

Awesome! You made it almost to the end. Now let's add the ability for the user to interact with our terminal, in writing.

In our `Terminal/index.tsx` will add the option to update the input, and also the commands that every word will execute. These are the most important things in a terminal, right?

Let's handle the user input in the `Terminal/index.tsx` file, and listen there when the user presses the `Enter` key. After that, we execute the function that he wants.

The `Terminal/index.tsx` the file will transform into:

```
import './terminal.css';
import {ForwardedRef, forwardRef, useCallback, useEffect, useRef, useState} from "react";
import {TerminalProps} from "../types";

export const Terminal = forwardRef(
  (props: TerminalProps, ref: ForwardedRef<HTMLDivElement>) => {
    const {
      history = [],
      promptLabel = '>',

      commands = {},
    } = props;

    const inputRef = useRef<HTMLInputElement>();
    const [input, setInputValue] = useState<string>('');

    /**
     * Focus on the input whenever we render the terminal or click in the terminal
     */
    useEffect(() => {
      inputRef.current?.focus();
    });

    const focusInput = useCallback(() => {
      inputRef.current?.focus();
    }, []);

    /**
     * When user types something, we update the input value
     */
    const handleInputChange = useCallback(
```

```

    (e: React.ChangeEvent<HTMLInputElement>) => {
      setInputValue(e.target.value);
    },
    []
  );

  /**
   * When user presses enter, we execute the command
   */
  const handleInputKeyDown = useCallback(
    (e: React.KeyboardEvent<HTMLInputElement>) => {
      if (e.key === 'Enter') {
        const commandToExecute = commands?.[input.toLowerCase()];
        if (commandToExecute) {
          commandToExecute?();
        }
        setInputValue('');
      }
    },
    [commands, input]
  );

  return (
    <div className="terminal" ref={ref} onClick={focusInput}>
      {history.map((line, index) => (
        <div className="terminal__line" key={`terminal-line-${index}-${line}`}>
          {line}
        </div>
      ))}
      <div className="terminal__prompt">
        <div className="terminal__prompt_label">{promptLabel}</div>
        <div className="terminal__prompt_input">
          <input
            type="text"
            value={input}
            onKeyDown={handleInputKeyDown}
            onChange={handleInputChange}
            // @ts-ignore
            ref={inputRef}
          />
        </div>
      </div>
    </div>
  );
});

```

As you can see, we have moved the type defined here, into the separate file we created called `types.ts`. Let's explore that file and see what new stuff we defined.

```
import {ReactNode} from "react";

export type TerminalHistoryItem = ReactNode | string;
export type TerminalHistory = TerminalHistoryItem[];
export type TerminalPushToHistoryWithDelayProps = {
  content: TerminalHistoryItem;
  delay?: number;
};

export type TerminalCommands = {
  [command: string]: () => void;
};

export type TerminalProps = {
  history: TerminalHistory;
  promptLabel?: TerminalHistoryItem;
  commands: TerminalCommands;
};
```

Good, now that we can execute commands from the terminal, let's define some in the `App.tsx`. We will create only two simple ones, but you let your imagination run wild!

```
import React, {useEffect, useMemo} from 'react';
import {Terminal} from "../Terminal";
import {useTerminal} from "../Terminal/hooks";

function App() {
  const {
    history,
    pushToHistory,
    setTerminalRef,
    resetTerminal,
  } = useTerminal();

  useEffect(() => {
    resetTerminal();

    pushToHistory(<>
      <div><strong>Welcome!</strong> to the terminal.</div>
      <div style={{fontSize: 20}}>It contains <span style={{color: 'yellow'}}>
```

```

<strong>HTML</strong></span>. Awesome, right?</div>
    <br/>
    <div>You can write: start or alert , to execute some commands.</div>
  </>
);
}, []);

const commands = useMemo(() => ({
  'start': async () => {
    await pushToHistory(<>
      <div>
        <strong>Starting</strong> the server... <span style={{color:
'green'}}>>Done</span>
      </div>
    </>);
  },
  'alert': async () => {
    alert('Hello!');
    await pushToHistory(<>
      <div>
        <strong>Alert</strong>
        <span style={{color: 'orange', marginLeft: 10}}>
          <strong>Shown in the browser</strong>
        </span>
      </div>
    </>);
  },
}), [pushToHistory]);

return (
  <div className="App">
    <Terminal
      history={history}
      ref={setTerminalRef}
      promptLabel={<>Write something awesome:</>}
      commands={commands}
    />
  </div>
);
}

export default App;

```


The constant `commands` defines the command that should be typed by the user. So if the user types `start` and hits enter, he will see the text “**Starting** the server... Done”.

What we've built

Thanks for reaching the end, **you are awesome!**

Here is what we've built together, hopefully, it will be helpful for your website or a client's website.



What's next?

What's next you ask? The terminal right now is pretty simple. You don't see the previously typed commands. If the user types an incorrect command, he will see nothing. And, of course, animations!

See you in part two for these new features.

Until then, you can [view the code on GitHub](#) or